In [1]:
```
#Concept: Norm

import numpy as np

A = np.array([2, 3])
print(np.linalg.norm(A)) # sqrt(13) ≈ 3.6055
```

```
3.605551275463989
```

In [2]:
```
#L2 Norm

import numpy as np

v = np.array([3, -4])
print(np.linalg.norm(v))          # ord=2 by default                      # 5.0
```

```
5.0
```

In [3]:
```
#L1 Norm
v = np.array([3, -4])
print(np.linalg.norm(v, ord=1))
```

```
7.0
```

In [4]:
```
#L∞ Norm

v = np.array([3, -4])
print(np.linalg.norm(v, ord=np.inf))                  # 4
```

```
4.0
```

In [ ]:

In [5]:
```
#concept: Vector addition

A = np.array([2, 3])
B = np.array([2, 3])
print(A + B)   # [4, 6]
```

```
[4 6]
```

In [6]:
```
#Concept: Scalar multiplication

S = 4
A = np.array([2, 3])
print(S * A)   # [8, 12]
```

```
[ 8 12]
```

In [7]:
```
#Concept: Inner product

v1 = np.array([1, 2])
v2 = np.array([3, 4])
dot = np.dot(v1, v2)
print(dot)   # 11
```

```
11
```

In [8]:
```
#Concept: Similarity

cos_sim = dot / (np.linalg.norm(v1) * np.linalg.norm(v2))
print("cosθ = ",cos_sim)   # 0.9839
```

```
cosθ =  0.9838699100999074
```

In [9]:
```
#Concept: Orthogonality

u = np.array([1, 0])
w = np.array([0, 1])
print(np.dot(u, w))          # 0 → orthogonal
```

```
0
```

In [10]:
```
#Concept: Projection

proj_v1_on_v2 = (np.dot(v1, v2) / np.dot(v2, v2)) * v2
print(proj_v1_on_v2)   # [1.32, 1.76] #Used in PCA and regression.
```

```
[1.32 1.76]
```

In [11]:
```
#Concept: Linear Independence

M = np.array([[1, 2], [3, 4]])
rank = np.linalg.matrix_rank(M)
print(rank)
```

```
2
```

In [12]:
```
#Concept: Basis

print(rank)
```

```
2
```

In [13]:
```
#Concept: Linear Combination
v1 = np.array([1, 2])
v2 = np.array([3, 4])
print(v1*5 + v2*3)   # [11, 16]
```

```
[14 22]
```

In [14]:
```
#Concept: Matrix add

M1 = np.array([[1, 2], [3, 4]])
M2 = np.array([[5, 6], [7, 8]])
print(M1 + M2)
```

```
[[ 6  8]
 [10 12]]
```

In [15]:
```
#Concept: Multiplication

M1 = np.array([[1, 2], [3, 4]])
M2 = np.array([[5, 6], [7, 8]])
print(M1 @ M2)
```

```
[[19 22]
 [43 50]]
```

In [16]:
```python
#Concept: Transpose
M1 = np.array([[1, 2], [3, 4]])
print(M1.T)
```

```
[[1 3]
 [2 4]]
```

In [17]:
```python
#Concept: Hadamard (element-wise)

M1 = np.array([[1, 2], [3, 4]])
M2 = np.array([[5, 6], [7, 8]])
print(M1 * M2)
```

```
[[ 5 12]
 [21 32]]
```

In [18]:
```python
#Concept: Determinant

M1 = np.array([[1, 2], [3, 4]])
det = np.linalg.det(M1)
print(det)
# det = -2.0 → Invertible
```

```
-2.0000000000000004
```

In [19]:
```python
#Concept: Identity Matrix

I = np.eye(2)
print(I)
# [[1. 0.]
#  [0. 1.]]
print(I @ M1)
# [[1. 2.]
#  [3. 4.]]
```

```
[[1. 0.]
 [0. 1.]]
[[1. 2.]
 [3. 4.]]
```

In [20]:
```python
#Concept: Inverse

M1 = np.array([[1, 2], [3, 4]])
print(np.linalg.inv(M1))
# [[-2.   1. ]
#  [ 1.5 -0.5]]
```

```
[[-2.   1. ]
 [ 1.5 -0.5]]
```

In [21]:
```python
#Concept: Rank

M1 = np.array([[1, 2], [3, 4]])
print(np.linalg.matrix_rank(M1))   # 2
```

```
2
```

In [22]:
```
#Concept: Trace

M1 = np.array([[1, 2], [3, 4]])
print(np.trace(M1))   # 5
```

```
5
```

In [23]:
```
#Concept: Symmetry

sym = np.array([[1, 2], [2, 3]])
print(np.allclose(sym, sym.T))   # True
```

```
True
```

In [24]:
```
#Concept: Orthogonal

Q = np.array([[0, 1], [1, 0]])   # reflection matrix
print(Q.T @ Q)
# [[1. 0.]
#  [0. 1.]]
```

```
[[1 0]
 [0 1]]
```

In [25]:
```
#Concept: Eigen

sym = np.array([[1, 2], [2, 3]])
eigvals, eigvecs = np.linalg.eig(sym)
print(f"Eigenvalues: {eigvals}")
print(f"Eigenvectors:\n{eigvecs}")
```

```
Eigenvalues: [-0.23606798  4.23606798]
Eigenvectors:
[[-0.85065081 -0.52573111]
 [ 0.52573111 -0.85065081]]
```

In [26]:
```
#Concept: PCA

data = np.array([[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3.0]])
mean = np.mean(data, axis=0)
centered = data - mean
cov = np.cov(centered.T)
eigvals, eigvecs = np.linalg.eig(cov)
print(eigvals)
```

```
[1.73707382 0.05392618]
```

In [27]:
```
#Concept: SVD

data = np.array([[2.5, 2.4], [0.5, 0.7], [2.2, 2.9], [1.9, 2.2], [3.1, 3.0]])
U, S, Vt = np.linalg.svd(data)
print(f"Singular values: {S}")
```

```
Singular values: [7.26731934 0.49605398]
```

In [28]:
```
#Concept: Derivative
```

```
from sympy import symbols, diff

x = symbols('x')
f = x**2 + 3*x
df = diff(f, x)
print(df)   # 2*x + 3
```

```
2*x + 3
```

In [29]:
```
#Concept: Partial derivative

y = symbols('y')
g = x**2 * y + y**3
partial_x = diff(g, x)
partial_y = diff(g, y)
print(f"∂g/∂x = {partial_x}")   # 2*x*y
print(f"∂g/∂y = {partial_y}")   # x**2 + 3*y**2
```

```
∂g/∂x = 2*x*y
∂g/∂y = x**2 + 3*y**2
```

In [30]:
```
#Concept: Gradient

from sympy import Matrix

grad = Matrix([[diff(g, x)], [diff(g, y)]])
print(grad)
# Matrix([[2*x*y], [x**2 + 3*y**2]])
```

```
Matrix([[2*x*y], [x**2 + 3*y**2]])
```

In [31]:
```
#Concept: Gradient descent

def f_gd(x):
    return (x-3)**2 + 2

def df_gd(x):
    return 2*(x-3)

x0 = 10.0
lr = 0.1
for i in range(20):
    x0 -= lr * df_gd(x0)
print(f"Minimum found at x ≈ {x0:.4f}")   # ≈ 3.0000
```

```
Minimum found at x ≈ 3.0807
```

In [32]:
```
import sympy as sp

#Concept: Lagrange multipliers

lam = sp.symbols('lambda')
x_l, y_l = sp.symbols('x y')
f_l = x_l + y_l                 # objective
g_l = x_l**2 + y_l**2 - 1       # constraint: unit circle
L = f_l - lam * g_l
eq1 = sp.diff(L, x_l)
eq2 = sp.diff(L, y_l)
```

```
eq3 = g_l
soln = sp.solve([eq1, eq2, eq3], [x_l, y_l, lam])
print("Stationary points:")
for s in soln[:2]:   # first two are real
    print(f"x = {s[0]}, y = {s[1]}, λ = {s[2]}")
```

```
Stationary points:
x = -sqrt(2)/2, y = -sqrt(2)/2, λ = -sqrt(2)/2
x = sqrt(2)/2, y = sqrt(2)/2, λ = sqrt(2)/2
```

In [33]:

```
#Concept: Lagrange multipliers

import sympy as sp
x = sp.symbols('x')

lam = sp.symbols('lambda')
y= sp.symbols('y')
f = x**2 + y**2
g = x + y - 1
L = f - lam*g
eqs = [sp.diff(L,x), sp.diff(L,y), sp.diff(L, lam)]
sp.solve(eqs, (x, y, lam))
```

Out[33]:

```
{lambda: 1, x: 1/2, y: 1/2}
```

In [34]:

```
import numpy as np

def lagrange_multiplier(x, y, m, n):
    return 1.0

# Demo Array
x = np.array([1, 2])
y = np.array([3, 4])
m = 2
n = 2
lagrange_multipler = lagrange_multiplier(x, y, m, n)
print(f"Lagrange Multiplier for 2D: {lagrange_multipler}")
```

```
Lagrange Multiplier for 2D: 1.0
```

In [ ]: